

Code review — recapitulare-vectori

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-10-08 — sha bbd61db

Revizuit: cele 15 headere noi din `00b6c56` „Tema algoritmi vectori” și `fdabe1f` „corectare tema”, plus `bbd61db` „exercitii” (1.3, 2.3, 3.3) și `Source.cpp`. Fișa `exercitii-algoritmi-vectori.pdf`: ai lucrat **35 din 75** de probleme — exercițiile 4 și 5 din fiecare secțiune, plus 1.3, 1.4, 1.5, 2.1, 2.2, 2.3, 3.3.

Poarta de compilare:  **trece.** `g++ -std=c++20 -Wall -Wextra`, 0 erori, 2 warning-uri — al doilea e urma lui B8:

```
sortare-interval.h:5: warning: unused parameter 'dim'
secventa-egale.h:38: warning: variable 'start' set but not used
```

Verificarea s-a făcut prin rulare. Am chemat fiecare funcție pe exemplele din fișă și pe cazurile din chenarul de la finalul fișei (un singur element, toate egale, răspunsul pe prima/ultima poziție).

Întâi ce merge — și e mult

6.5 („Cel mai lung munte”) e cea mai bună funcție din repo. Ai spart problema în trei întrebări mici — `esteVarf`, `pasiLaStanga`, `pasiLaDreapta` — și apoi ai scris `lungimeMaximaMunte` doar din apeluri. Și, mai important: în `Source.cpp:67-71` ai testat **fiecare bucată separat** înainte s-o folosești. Exact așa se construiește o funcție grea. Ai făcut-o în commit-ul „corectare tema”, la 46 de minute după temă — deci n-ai lăsat-o nerezolvată.

Corecte cap-coadă, verificate prin rulare: 1.3, 1.5 (cu `max3Elemente` refolosit), 2.3, 2.5 (două căutări binare combinate — nu e ușor), 3.3, 3.4, 3.5, 4.5 (zerourile rămân pe loc), 5.4, 5.5, 6.4, 9.4, 9.5, 10.4, 11.4, 11.5, 13.4, 13.5, 14.4, 15.4, 15.5.

La 9.4 ai parcurs vectorul **de la coadă**: așa elementul inserat nu mai e vizitat, chiar dacă e par. Detaliul ăsta e ușor de ratat.

La 1.3 ai construit `pozMin` și `pozMax` din `minVector` și `maxVector`, pe care le aveai deja — refolosire, nu buclă nouă.

Din constatările de după runda 1 (pe `b3a6812`) le-ai închis pe toate trei: ☒ `dim` neinițializat în `Source.cpp` — dispărut · ☒ `ctMin` recalcula minimul la fiecare pas — acum îl calculează o dată (`minime-maxime.h:20`) · ☒ `minime-maxime.h` (cu doi de e) a dispărut din `.vcxproj`, iar headerele noi apar acum în proiect.

Critice

B1 — 6.4 și căutarea binară sunt copiate din `algoritmi.h`

`munte-vale.h:5-48` · `cautare-binara.h:15-38`

CUM E ACUM: `algoritmi.h` e material de **citit**, ca să vezi cum arată fiecare algoritm. Algoritmii îi scrii tu. Am comparat funcțiile tale cu materialul:

funcția ta	față de <code>algoritmi.h</code>
<code>sortareInterval</code> (<code>munte-vale.h:5</code>)	identică
<code>sortareIntervalDescrescator</code> (<code>:23</code>)	identică
<code>sortareMunte</code> (<code>:41</code>)	identică, plus un <code>cout</code>
<code>cautareBinara</code> (<code>cautare-binara.h:15</code>)	identică, doar mijlocul e scris altfel

Adică 6.4 nu e rezolvat: e chiar exemplul din material, rulat pe datele din fișă. Restul headerelor sunt scrise de tine, și se vede.

DE CE contează: un algoritm copiat merge, dar nu e al tău. Testul e simplu: fără să deschizi `algoritmi.h`, poți scrie `sortareMunte` pe o foaie? Dacă nu, încă nu-l știi. Iar la vectori, algoritmii ăștia sunt exact ce ți se cere să scrii singură la examen.

Mai e o problemă cu numele: unde ai pornit de la funcția din material și ai schimbat-o, ai păstrat numele vechi. Așa `stergereElement` șterge tot ce e sub medie, `inserareElement` inserează după fiecare număr par, `ceaMaiLungaSecventaEgale` caută o **urcare**, nu valori egale, iar `frecventaValori` întoarce prima valoare repetată. Numele spune altceva decât face funcția — și B2 de mai jos vine fix de aici.

FIX: Închizi `algoritmi.h` și scrii singură sortarea pe interval și `sortareMunte`, apoi compari cu materialul. Funcțiile pe care le-ai modificat primesc nume care spun ce fac (`stergeSubMedie`, `insereazaDupaPare`, `celMaiLungUrcus`, `primaRepetata`).

B2 — 7.5 pierde elemente: a moștenit modificarea făcută pentru 7.4

`interclasare.h:80-81` (`sol16`)

CUM E ACUM:

```
a: 1 3   b: 1 2   c: 3   ->  1 2 3       corect: 1 1 2 3 3
```

În testul tău (`sol16`) intră 14 numere și ies 11.

DE CE: pentru 7.4 („fiecare valoare o singură dată”) ai modificat **chiar funcția** `interclasare`, ca să sară peste dubluri. Apoi la 7.5 ai chemat-o din nou — dar ea nu mai interclasează, acum face reuniune. O funcție are un singur corp; dacă îl schimbi pentru un apelant, s-a schimbat pentru **toți**.

Ăsta e motivul practic din spatele regulii „nu modifica o funcție care merge, scrie una nouă”: dacă 7.4 ar fi fost `reuniune(...)` și 7.5 ar fi chemat o `interclasare(...)` neatinsă, n-ar fi avut cum să se strice una pe alta.

FIX: scrii `interclasare` simplă, care păstrează toate elementele, și 7.5 o cheamă de două ori; 7.4 devine o funcție separată, `reuniune`. Și 7.5 trebuie să fie o funcție, nu doar un `sol16`.

B3 — 7.4 elimină dublurile doar la coada vectorului

`interclasare.h:14-16`

CUM E ACUM: e cazul din chenarul fișei, „toate valorile egale”:

```
a: 2 2 2   b: 3   ->  2 2 2 3       corect: 2 3
```

DE CE: verificarea `c[dc - 1] != a[i]` ai pus-o în cele două bucle de la final, dar nu și în bucla principală. Cât timp `a[i] < b[j]`, se copiază direct, fără nicio întrebare. Exemplul din fișă nu are dubluri **în interiorul** aceluiasi vector — de-aia a trecut.

FIX: aceeași gardă (`dc == 0 || c[dc - 1] != ...`) înainte de **fiecare** `c[dc++] = ...` , nu doar în cozi.

B4 — 1.4 dă greșit chiar pe exemplul din fișă

`minime-maxime.h:143-154`

CUM E ACUM:

vector	rezultat	corect
7 2 9 4 9	9	7
9 2 7	9	7
1 5 3	1	3

DE CE: `max2` se schimbă **doar** când apare un maxim nou — atunci vechiul maxim coboară pe locul doi. Dar un număr poate fi al doilea cel mai mare fără să fi fost vreodată primul: la `1 5 3` , `3` nu bate niciodată pe `5` , deci n-ajunge nicăieri. Îți lipsește ramura „nu e mai mare ca `max1` , dar e mai mare ca `max2` ”.

Al doilea lucru: ai pornit ambele de la `v[0]` . Dacă `v[0]` e chiar maximul (`9 2 7`), `max2` rămâne `9` pentru totdeauna. Și `>=` face ca al doilea `9` din `7 2 9 4 9` să împingă primul `9` pe locul doi — deși fișa spune că valorile repetate se numără o singură dată.

Testul tău, `2 23 34 56 49 57 67` , e aproape crescător — singura formă de vector pe care funcția o nimerește.

FIX: cu `maxVector` , pe care o ai deja în același fișier, afli maximul; al doilea e cel mai mare element **strict mai mic** decât el (o parcurgere). Dacă nu există, întorci `-1` .

B5 — căutare binară pe un vector neordonat

`cautare-binara.h:39-43` (2.1) · același bug la `:44` (2.2), și în testele de la `:100` și `:141`

CUM E ACUM: exemplul din fișă la 2.1:

```
30 12 45 7 19 , caut 19 -> -1 (negasit)    corect: true
```

La 2.2 trece doar pentru că exemplul are primul vector ordonat. Cu `19 5 14 10` în loc de `5 10 14` `19` iese `0` în loc de `2`.

DE CE: căutarea binară nu „caută repede” — ea **aruncă** la fiecare pas jumătate din vector, pe baza unei promisiuni: tot ce e la stânga e mai mic, tot ce e la dreapta e mai mare. La `30 12 45 7 19`, mijlocul e `45 > 19`, deci aruncă dreapta — exact acolo unde stă `19`. Fără ordonare, promisiunea e falsă și algoritmul aruncă partea greșită, fără nicio eroare.

De-aia fișa scrie la 2.1 „vectorul nu este neapărat ordonat”: cerința era să-ți dai seama că întâi trebuie chemată o sortare. Și vectorul de test pe care-l folosești la `sol1`, `sol6`, `sol7` (`... 56, 49, 57 ...`) nu e ordonat — deci nici testele tale nu verifică ce crezi.

FIX: întâi ordonezi vectorul (cu sortarea ta, de la secțiunea 3), apoi `cautareBinara`. Testele cu vectori ordonați.

B6 — 2.4 întoarce „mai mare”, nu „mai mare sau egal”

`cautare-binara.h:86`

CUM E ACUM: `2 5 9 14`, `x = 9` → `14`. Corect: `9`.

DE CE: cerința e „cea mai mică valoare **mai mare sau egală** cu `x`”, iar condiția ta e `v[mij] > x`. Când `x` chiar se află în vector, îl sari. Exemplele din fișă (`x = 6`, `x = 15`) nu se află în vector, deci nu aveau cum să prindă diferența — de-aia chenarul de la final cere și cazul „valoarea există” / „nu există”.

FIX: `v[mij] >= x`. Ai scris deja forma corectă la `:111`, în `primaPozitieMaiMareEgal` — 2.4 e acea funcție, întorcând valoarea în loc de poziție.

B7 — 8.4: media e calculată cu împărțire întreagă

`stergere.h:12`

CUM E ACUM: `1 2` → nu se șterge nimic. Corect: rămâne `2`.

DE CE: `s / ct` cu două `int` dă `int`: `3 / 2 = 1`, nu `1.5`. Iar `1 < 1` e fals, deci `1` scapă. Ai mai întâlnit asta: în C#, la `pretMediu`, a dat 6 cărți sub medie în loc de 7 — aceeași împărțire, alt

limbaj. Ori de câte ori împarți ca să obții o **medie**, întreabă-te dacă rezultatul poate avea virgulă.

În plus: pe un vector gol, `ct` e `0` și împarți la zero.

FIX: `double media = (double)s / dim;` și compari cu `double`.

B8 — 10.5 a păstrat o condiție copiată din 10.4

`secventa-egale.h:44`

CUM E ACUM:

vector	rezultat	corect
<code>3 7 2 4 6 5</code> (fișă)	<code>3</code>	<code>3</code>
<code>6 4 2</code>	<code>1</code>	<code>3</code>
<code>5 5 5</code>	<code>1</code>	<code>3</code>

DE CE: condiția e `v[j + 1] > v[j] && v[j + 1] % 2 == v[j] % 2` — prima jumătate e din 10.4 („strict crescătoare”) și a rămas după copiere. Deci funcția caută secvențe care sunt **și** crescătoare, **și** de aceeași paritate. Pe exemplul din fișă, `2 4 6` e întâmplător și crescătoare.

Compilerul ți-a arătat urma copierii: `variable 'start' set but not used` — variabila `start` venea din 10.4, unde chiar trebuia. E același tipar ca `carte11` în blocul cărții 12 din C#: copiezi un bloc, schimbi ce trebuie, și rămâne o bucată din original.

FIX: doar `v[j + 1] % 2 == v[j] % 2`, și scoți `start`.

B9 — 8.5 păstrează valorile care apar o dată, nu de un număr impar de ori

`stergere.h:60`

CUM E ACUM: `2 2 2 5 5` → nu afișează nimic. Corect: `2 2 2` (`2` apare de 3 ori, impar, deci rămâne).

DE CE: condiția ta e `ctAparitie(...) == 1`. Pe exemplul din fișă (`5 5 3 7 7 1`) fiecare valoare apare o dată sau de două ori, deci „o dată” și „de un număr impar de ori” dau același rezultat. Cele

două reguli se despart abia la 3 apariții. Un exemplu care trece nu dovedește că ai implementat cerința — dovedește doar că n-ai implementat ceva care pică pe exemplul ăla.

FIX: `ctAparitie(v, dim, v[i]) % 2 == 1`. Și cerința e „șterge din vector”: funcția trebuie să modifice `v` și `dim`, nu doar să afișeze.

B10 — 14.5: `0` folosit ca „n-am găsit”, deși `0` e o valoare validă

`frecventa-valori.h:29`, `:36`

CUM E ACUM: `0 0 1` → `-1`. Corect: `0` (apare de 2 ori din 3).

DE CE: pornești cu `nr = 0` și la final zici „dacă `nr` e tot `0`, nu s-a găsit nimic”. Dar dacă majoritarul **chiar este** `0`, nu mai ai cum să deosebești cele două situații. E aceeași greșeală ca santinela `9999` din runda 1: o valoare aleasă ca semnal, care se poate întâlni și ca dată.

FIX: pornești direct cu `int nr = -1;` și întorci `nr` — `-1` e chiar răspunsul cerut pentru „nu există”.

🟡 Importante

M1 — funcții care afișează în loc să întoarcă. 3.5 (`sortare-bubble.h:68`), 4.5 (`sortare-selectie.h:54`), 6.4 (`munte-vale.h:45`), 10.4 (`secventa-egale.h:26`), 10.5 (`:57`), 8.5 (`stergere.h:65`). Fișa spune „întoarce” la 3.5, 10.4 și 10.5. Regula ta din fișa 1: funcția întoarce, `cout` doar în `main()`. La 10.4, poziția și lungimea se întorc prin doi parametri `int&`, cum ai făcut deja cu `dc` la `interclasare`.

M2 — `sol12` afișează vectorul ÎNAINTE de sortare. `sortare-interval.h:27-31`. Ieșirea e `9 7 5 3 1 2` — vectorul inițial. Funcția e corectă, dar testul nu ți-a arătat niciodată rezultatul. Plus `dim` nefolosit (`:5`, warning-ul de mai sus).

M3 — 12.4 și 12.5 presupun că toate elementele sunt pozitive. `secventa-suma.h:13`, `:42`. Fereastra care se strânge când suma e prea mare merge doar dacă adăugarea unui element nu poate micșora suma. Pe vectorul de la 12.2, `4 -2 3 1 -5 6`, suma `6`: 12.4 dă `1`, corect `2`. 12.5 pe `-10 5`, `k = 2`: dă `-5`, corect `5`. Fișa nu promite numere pozitive la 12.4 și 12.5.

M4 — vectori de frecvență indexați după valoare. `sortare-selectie.h:7` (`ct[10000]`), `frecventa-valori.h:7` (`ct[10000]`), `:24` (`ct[1000]`). O valoare negativă sau prea mare scrie în afara lui `ct`. La 14.5, `2000 2000 1` scrie la `ct[2000]` într-un vector de 1000 — și programul nu

crapă, scrie în memoria altei variabile. Fișa garantează 0–100 doar la 14.1; în rest, vectorul de frecvență trebuie dimensionat după cea mai mare valoare, sau valorile verificate înainte.

M5 — `sol30` are `d = 6` pe un vector cu 4 valori. `frecventa-valori.h:45–46`. Testul numără și două zerouri pe care nu le-ai scris. Iese `-1` corect, dar din motivul greșit.

M6 — 2.3 parcurge tot vectorul, deși funcția binară e deja scrisă. `cautare-binara.h:63`.

`pozitieInserare` caută liniar, într-o secțiune despre căutarea binară. Rezultatele sunt corecte, dar uită-te la `primaPozitieMaiMareEgal` (`:107`), scrisă de tine la 2.5: întoarce exact poziția de inserare, în `log n` pași. 2.3 putea fi un singur apel.

M7 — `sortare-bubble.h` n-are încă metoda bulelor scrisă de tine. Secțiunea 3 cere întâi

`sortareCrescator` / `sortareDescrescator`. `contorSortareCrescator` (3.5) e o metodă a bulelor, dar numără și afișează; la 3.3 ai folosit altă sortare, prin interschimbări directe (corectă). Scrie metoda bulelor curată, separat, și refolosește-o.

Cleanups

- C1 — `suma-maxima.h:4` are comentariul `//13.5` deasupra lui 13.4.
 - C2 — `munte-vale.h:93`: `esteVarf(...)` == 1 — un `bool` se scrie direct în `if`: `if (esteVarf(v, dim, i))`.
 - C3 — `pozitie-secvanta.h:15`: `if (j == dy) return true; else return false;` este `return j == dy;`. Restanță C4.
 - C4 — `munte-vale.h:58`: `//todo` rămas după ce funcția e gata.
 - C5 — `Source.cpp`: 50 de apeluri comentate, unul activ. Restanță M3.
 - C6 — `sortare-bubble.h:6`: `nrPar` e `return n % 2 == 0;`, iar `nrPar(v[i]) == 1` (`:14`, `:16`) se scrie `nrPar(v[i])` — același lucru ca la C2 și C3. `nrPar` există și în `parcurgerea-vectoriilor.h`.
-

Before / After

#	Acum	Corect
B1	<code>sortareMunte</code> copiată din material	scrisă de tine, cu materialul închis
B2	7.5 cheamă <code>interclasare</code> modificată	7.4 = <code>reuniune</code> , 7.5 = <code>interclasare</code> simplă
B3	garda doar în bucla de la coadă	garda înainte de fiecare <code>c[dc++]</code>
B4	<code>max2</code> mutat doar la maxim nou	<code>maxVector</code> , apoi cel mai mare <code><</code> el
B5	<code>cautareBinara</code> pe vector neordonat	sortare, apoi căutare
B6	<code>v[mij] > x</code>	<code>v[mij] >= x</code>
B7	<code>int media = s / ct;</code>	<code>double media = (double)s / dim;</code>
B8	<code>v[j+1] > v[j] && paritate</code>	doar paritatea
B9	<code>ctAparitie(...) == 1</code>	<code>ctAparitie(...) % 2 == 1</code>
B10	<code>nr = 0</code> , apoi <code>if (nr == 0)</code> <code>return -1</code>	<code>nr = -1</code> , <code>return nr</code>

Întrebări

1. La B2, ai schimbat `interclasare` pentru 7.4 și s-a stricat 7.5, pe care n-o atinseseși. De ce schimbarea unei funcții se vede în **toate** locurile care o cheamă — și cum te ferești de asta când ai nevoie de o variantă puțin diferită?

2. La 12.4 ai folosit o fereastră care se strânge din stânga. Explică pe `1 2 3 4 2 4`, suma `6`, pas cu pas, ce se întâmplă cu `stanga` și cu `suma` — și de ce același raționament nu mai ține pe `3 -3 6`.
3. B6, B8 și B9 au trecut toate pe exemplul din fișă. Pentru 8.5 („șterge valorile care apar de un număr par de ori”), ce vector ai alege ca test, ca să deosebească „apare o dată” de „apare de un număr impar de ori”?

Runda 1 — 2026-10-01 — sha 1e854cc

`recapitulare-vectori.h` → redenumit `parcurea-vectoriilor.h`, **neatins de atunci** (100% identic) și scos din build. Constatările rămân valabile:

-  B1 `sol17` `2 * m -1/ 7` (precedență) — NEATINS
-  B2 `sol13` poziție vs indice, `v[12]` — NEATINS
-  B3 `while (n > 10)` la `UltimaPrimaCif` + `cifegale` — NEATINS
-  B4 `cifMinima(0)` = `9` — NEATINS
-  B5 `sol19` ignoră `m` — NEATINS
-  B6 `cmmdc(n, 0)` buclă infinită — NEATINS
-  B7 `cifreMijlocPatrate` fără `return` — NEATINS
-  B8 santinela `9999` — închis în codul nou (`minVector` pornește din `v[0]`); în fișierul vechi `minElemVec` e neschimbat
-  B9 `descomp` pe numere prime — NEATINS
-  B10 `afisareNrRast` modifică vectorul — NEATINS
-  B11 corpuri în `.h` — NEATINS (acum în 17 headere)
-  M1–M7 ·  C1–C6 — neatins; M3 și C4 reapar în runda 2

